UDA-Bench NBA experiments

E1 graph structure, X6 coupling ablation, schema-based extractors

October 15, 2026

1 Goal

Three studies on the same corpus, with the same language model:

- **E1, graph structure.** WUKONG, LightRAG, GraphRAG and KGGen build a graph from the corpus. We measure how the graphs can be navigated and queried, with statistics that can be read without knowing the domain: size, node and relation vocabularies, edge direction and properties. Recall against the gold tables is a control, to check that the graphs contain comparable information.
- **X6, coupling ablation.** WUKONG with six schemas, from the schema core alone (types, fields, endpoints) to the full extraction schema, adding one family of annotations at a time.
- **Schema-based extractors.** Three tools that take a schema as input (Neo4j's `neo4j-graphrag`, LlamaIndex's `SchemaLLMPathExtractor`, OntoGPT's SPIRES), given our schema in their own formats, scored like the X6 variants.

2 Common setup

Corpus and gold. The NBA dataset of UDA-Bench: 216 English Wikipedia articles, 141 about players, 30 about teams, 29 about cities and 16 about team owners. The gold tables (player, team, city, manager) have one row per article; the row ID is the document number.

Model. Every system uses `gpt-5.6-luna` with low reasoning effort, and `text-embedding-3-small` where it embeds. Output tokens below include reasoning tokens.

WUKONG. Engine at commit `baeeb13`, chunks of 800 tokens with 120 overlap, document level reading the first 8000 tokens of each document, export to Neo4j CSV (`x6-config.toml`). The schema used for every result outside the ablation is the revised full schema (`workspace-x6-s5/`, variant s5 of Section 4): four entity types (Player, with a document-level `current_team` field; Team; City; Owner) with typed fields and three relationship types (PlaysFor, with a `tenure` field; LocatedIn; Owns, with a `since_year` field). Each WUKONG configuration was run three times; tables give the mean and, where runs differ, the range.

Schema hygiene. The first version of the schema (`workspace/`) used example values taken from the corpus, including gold answers (two players' birth dates, two cities' states, a team's owner, and subject names), and one instruction used a subject team as its format example. We replaced every such value by one that is neither a gold cell nor the name of a subject entity, leaving everything else unchanged. `make_schema_variants.py` and `make_coupling_variants.py` reject any schema whose examples, descriptions or instructions contain a gold value or a subject's name. Two conventions of the gold tables are used on purpose as target-schema knowledge: the closed vocabulary {Frontcourt, Backcourt} for a player's position, and the date format `YYYY/M/D`.

Matching against gold. Each gold row is matched to graph nodes by the article's title and the gold name, after normalization (case, accents, punctuation), and by aliases: for people, the same surname with a compatible first name (prefix or common nickname); for teams, the nickname alone; for cities, *City, State* and *City of X*. Nodes of typed graphs (WUKONG and the schema-based extractors) must also have the right type (`e1_analysis.py`).

3 E1: graph structure

Systems.

- **WUKONG**, revised full schema (s5), three runs.
- **LightRAG** 1.5.7, default extraction, chunks of 1200 tokens with 100 overlap, 16 concurrent LLM calls (`lightrag_build.py`).
- **GraphRAG** 3.2.0, standard indexing with its default entity types (organization, person, geo, event), one gleaning, chunks of 1200 tokens with 100 overlap, default prompts (`graphrag-config/`).
- **KGGen**, GitHub commit `6259b4c` (March 2026) in the environment of its own lock file (`baselines/kggen_build.py`). The latest release, 0.4.0, clusters by putting every remaining entity of the graph in one prompt, which cannot hold this corpus; the commit replaces it with a deduplication built for large graphs. Pipeline of its README: one graph per document (whole article per call, its default; entities, then relations, then SemHash deduplication), aggregated, then deduplicated over the corpus with its default method (SemHash, then k-means groups of 128 items and one LLM call per item to find its duplicates within the group). Temperature 1.0 and at most 16,000 tokens, as KGGen requires for this model family; response cache disabled.

LightRAG, GraphRAG and KGGen receive no schema. KGGen's only other input, a context string, gets the corpus domain statement of the WUKONG schema; at this commit the code passes it to no prompt. KGGen is a triple extractor that canonicalizes its entity and relation names after extraction, so it tests whether label clustering alone yields a queryable vocabulary.

Build cost. Table 1. Runs were executed concurrently with other runs, so wall times are indicative.

Graph. For WUKONG we count domain nodes and edges; the provenance nodes and edges that link every object to its source chunk and document are left out. For LightRAG we read its GraphML graph; for GraphRAG its entity and relationship tables; for KGGen its triples after the corpus deduplication. KGGen entities are bare strings: an entity that occurs in no triple is an isolated node (9,251 of them).

	LLM calls	Input tokens	Output tokens	Wall time
WUKONG	4,447	6.4M	1.1M	18 min
LightRAG	6,341	17.3M	7.8M	92 min
GraphRAG	24,965	35.3M	15.1M	101 min
KGGen	26,025	13.8M	1.7M	228 min

Table 1: Cost of building each graph over the 216 documents. GraphRAG's calls are 3,362 graph extractions, 16,511 description summaries and 5,092 community reports; it also used 8.2M embedding tokens. KGGen's are 440 extraction calls (14 min) and 25,585 deduplication calls (214 min), plus 0.1M embedding tokens; it ran alone.

Relation vocabulary, own labels. The label of an edge is what the system stores: the relationship type in WUKONG, the set of keywords in LightRAG, the predicate of the triple (after its deduplication) in KGGen. GraphRAG stores no relation label, only a free-text description and a weight. For a labeling we report the number of distinct labels, the fraction used by a single edge, the number of most frequent labels needed to cover 50% and 90% of the edges, and the mean number of distinct labels per pair of node types (over pairs with at least 100 edges).

Relation vocabulary, derived predicates. To measure LightRAG, GraphRAG and KGGen in the same way, and to give GraphRAG a label at all, an LLM (same model and effort as the builds) reads the description of every edge, together with the names of its two endpoints, and names the main relation as a short predicate in lower snake case, 1–3 words, present tense, without dates, numbers or names, and states which endpoint is its subject (`derive_labels.py`; the prompt's examples come from an unrelated domain). For KGGen the description is the triple itself. Edges are labeled independently, 100 per call: 980 calls, 5.5M input and 1.5M output tokens for the 97,931 LightRAG and GraphRAG edges, and 205 calls, 0.8M input and 0.3M output tokens for the 20,475 KGGen edges; every edge received a label.

Synonym merging. To discount surface variation, all labels are embedded (`text-embedding-3-small`, 512 dimensions) and merged by leader clustering: in decreasing order of frequency, each label joins the most similar existing leader if the cosine similarity is at least t, and otherwise becomes a leader (`relation_vocab.py`). We report $t = 0.85$. At $t = 0.80$ the largest cluster already merges opposite relations (`plays_for` with `plays_against`), so 0.85 is the most aggressive threshold that still merges only synonyms; lower thresholds are in `relation_vocab.json`.

Direction. For a derived predicate, an edge is stored against its relation when the subject of the predicate is the edge's target. We also report the fraction of predicates with at least 10 edges where at least 10% of the edges go each way. In WUKONG direction is fixed by the endpoint types of each relationship type. LightRAG stores an undirected graph.

Properties. Typed property values per node, and the fraction of nodes whose content is only free text (a description). KGGen nodes carry neither: attribute values are objects of triples (*Boston Celtics, has won, 18 NBA championships*).

Control: recall against gold. A gold attribute value (non-empty, and non-zero for counts) is found in WUKONG if the typed field holds it (numbers within 1%, city areas in either square miles

or km^2, since the gold table mixes them). For LightRAG and GraphRAG it is found if it appears anywhere in the descriptions of the matched nodes or of their edges, and for KGGen in the text of their triples: dates in the usual written forms, counts only within 60 characters of a word naming what is counted, positions through *forward/center* and *guard*. This is an upper bound: the value is in the graph but not in a form a query can select. A gold relation (player's team, team's city, team's owners, owner's team) is found in WUKONG if an edge of the right type joins the matched nodes, and in LightRAG, GraphRAG and KGGen if any edge joins them.

		WUKONG	LightRAG	GraphRAG	KGGen
Size	Nodes	2,134	34,162	37,917	21,429
	Edges	3,212	44,040	53,891	20,475
Node vocabulary	Node types	4	14	99	none
	Node types used once	0	1	94	–
Relation vocabulary, own labels	Labels	3	25,151	none	4,496
	Labels used once	0%	79%	–	66%
	Labels to cover 90% of edges	1	20,747	–	2,449
Relation vocabulary, derived predicates	Labels	3	4,388	4,428	2,946
	Labels used once	0%	56%	52%	55%
	Labels to cover 50 / 90% of edges	1 / 1	51 / 1,079	47 / 936	67 / 1,113
	Labels per node-type pair (mean)	1	310	736	–
Synonyms merged ($\cos \geq 0.85$)	Labels	3	3,488	3,477	2,306
	Labels to cover 90% of edges	1	738	633	738
Direction	Edges stored against their relation	0%	48%[a]	16%	5%
	Frequent labels used in both directions	0%	85%	56%	17%
Properties	Typed property values per node	1.5	0	0	0
	Nodes described only by free text	0%	100%	100%	–[d]
Control: recall vs. gold	Entities found	214/216	214/216	214/216	212/216
	Attribute values found[b]	871/955	714/955	729/955	488/955
	Relations found[c]	171/220	195/220	192/220	188/220

Table 2: E1: structure of the four graphs. WUKONG (schema s5) values are means of three runs, which range 2,118–2,152 nodes, 3,161–3,284 edges, 866–873 attribute values and 170–172 relations, and find the same 214 entities. For WUKONG the derived and merged rows repeat its own labels. [a]LightRAG's graph is undirected. [b]For LightRAG, GraphRAG and KGGen, a value counts if it appears in their text (upper bound). [c]For LightRAG, GraphRAG and KGGen, any edge between the two entities counts, whatever relation it describes; most of their extra relations are players' teams outside the NBA, which the WUKONG schema does not target. [d]KGGen nodes are bare strings, with neither properties nor descriptions.

Notes.

- The three graphs find the same entities and comparable numbers of facts. WUKONG finds more attribute values as typed fields than the baselines hold anywhere in their text; the baselines connect more player–team pairs, mostly with non-NBA clubs.
- In the derived labelings, about 50 predicates cover half of the edges and about 1,000 are needed for 90%; more than half of all predicates label a single edge. Merging synonyms at $t = 0.85$ reduces the 90% figure to 738 and 633.

- KGGen's own canonicalization does not change this. After its deduplication it still uses 4,496 predicates, two thirds of them on a single edge, and needs 2,449 to cover 90% of the edges; derived and merged as for the RAG graphs, it needs 738, as many as LightRAG. Its edges are directed (5% stored against their relation), but its graph has no node types and no properties, and holds the fewest attribute values (488).
- Example: in LightRAG, the 892 edges between a player and a team in the gold tables carry 676 distinct keyword labels; *played for* labels 17 of them.

4 X6: coupling ablation

Variants. Every key of the full schema belongs to exactly one family; variant sk keeps the core and the first k families (`make_coupling_variants.py`, which checks that s4 equals the full schema):

- **s0, core:** types and their descriptions, fields (data type, description), the primary key of each entity type (the engine requires one), relationship endpoints, the corpus language. No source is bound: every entity and relationship type is extracted from every chunk of every collection, never at document level, and no relationship identity is declared.
- **s1, +bindings:** which collection and context level each type and field is found at, and the context levels of each endpoint.
- **s2, +guidance:** the corpus domain statement, instructions and examples.
- **s3, +validation:** options, regex, required fields.
- **s4, +identity (full schema):** relationship deduplication policies and merge strategies.
- **s5, +current_team:** a revision of the schema made after seeing the s4 results, not an annotation family. In s4 a player's current team is the `tenure` of PlaysFor, decided by every chunk that mentions the player and a team; a chunk sees only part of a career, so players get on average about two teams marked *latest*. A player's article *is* the player, so s5 adds a Player field `current_team`, read once at document level from the player's own article and skipped in chunks. Everything else is as in s4.

Measures. On the 216 subject entities: entities found, and entities split into several nodes. A gold attribute value is *exact* if the typed field holds it in the gold's form, so a query can compare and filter it as it is, and *lenient* if it holds it in any form ("December 30, 1984" for 1984/12/30, "third pick" for 3, "power forward" for Frontcourt); a value is *wrong* if the field is filled but not leniently equal. Relations are the 148 gold relations between gold entities, found when an edge of the right type joins the matched nodes. For the current team of the 90 players whose gold team is an NBA team we report how many are found, and the precision of what is marked as current: every team on a PlaysFor edge with tenure *latest*, or the value of `current_team` in s5 (`x6_analysis.py`).

Notes.

- Without guidance, most attribute values are found but not in a form a query can use: 763 lenient against 449 exact in s0. Guidance closes the gap (861 exact against 864 lenient in s2) and reduces split entities from 47 to 9.

	s0 core	s1 +bindings	s2 +guidance	s3 +validation	s4 full +identity	s5 +current_team
LLM calls	4,011	4,352	4,442	4,444	4,443	4,447
Output tokens	2.44M	1.02M	1.08M	1.10M	1.10M	1.11M
Wall time (min)	30	17	19	18	18	18
Edges	8,919	6,809	5,905	5,149	3,339	3,212
Entities found (of 216)	205	208	213	214	213	214
Entities split	62	47	9	8	8	8
Attribute values, exact (of 955)	449	467	861	870	865	871
Attribute values, lenient	763	801	864	870	865	871
Wrong values	47	24	22	19	21	21
Relations found (of 148)	133	136	138	136	135	136
Current team, edges: found (of 90)	18	12	77	78	78	78
precision	85%	75%	53%	46%	46%	58%
Current team, field: found (of 90)	–	–	–	–	–	77
precision	–	–	–	–	–	90%

Table 3: X6: WUKONG with each schema variant, means of three runs. Over the three runs of a variant, the values vary by at most 10 split entities, 15 attribute values, 3 relations, 2 current teams found and 174 edges, and the current-team precision by at most 6 points, except 18 points in s0, which marks few teams; the ranges are in `runs/x6/summary.csv`. Wall time excludes one run per variant from s2 to s5, during which the computer slept.

- Bindings halve the output tokens (2.44M to 1.02M), shorten the build from 30 to 17 minutes and halve the wrong values.
- Identity removes repeated edges (5,149 to 3,339) without changing the scores.
- The current team marked on edges has low precision once guidance asks for it (46–53%). Reading it as a document-level field of the player keeps the same recall and raises precision to 90%.

5 Schema-based extractors

Translation rule. Each tool receives our schema in its own format: every key of the full schema goes into the tool's matching slot if it has one, even if the tool then ignores it, and never into a different slot. Table 4 summarizes the mapping; each tool's `mapping.md` lists every key. Each tool keeps its default chunking and pipeline; a small wrapper around its LLM client sets the model and reasoning effort and records token usage. All three completed every document without a failed call (`baselines/`).

Neo4j `neo4j-graphrag` 1.22.0. The components of `SimpleKGPipeline` without the database writer: fixed-size splitter (4,000 characters, 200 overlap), LLM extractor with structured output, schema pruning, and its default entity resolution (per type, exact match on `name`, first property values win, duplicate relationships merged), reimplemented in Python because it only runs inside Neo4j with APOC. The chunk embedder is skipped; it does not affect extraction.

LlamaIndex `SchemaLLMPathExtractor` 0.14.25. With this model and effort the stock extractor cannot fill properties: its open property dictionary is either rejected by strict structured output

Schema element	Neo4j	LlamaIndex	OntoGPT
Types, endpoints	types, patterns	names, triples	classes, slots
Type descriptions	yes	no slot	yes*
Field descriptions	yes	flat list over types	yes
Options	no slot	no slot	enum
Regex	no slot	no slot	pattern†
Instructions	no slot	no slot	prompt annotation*
Examples	no slot	no slot	examples*
Primary key, required	KEY, EXISTENCE	entity name	key, required
Bindings	no slot	no slot	a root class per article type
Identity	exact name per type	exact name	identical id
Domain statement	no slot	no slot	schema description*
Unit of extraction	4,000 characters	1,024 tokens	whole document

Table 4: Where each element of the schema goes. *In the slot, but OntoGPT 1.2.0 does not put it in the prompt. †In the template but not enforced: enforced, a single non-matching value rejects the whole document, and 0 of 8 pilot documents survived.

or forced to be empty. We use its documented `kg_schema_cls` parameter with the stock models, properties typed as optional strings. Defaults otherwise: sentence splitter (1,024 tokens, 20 overlap), at most 10 triplets per chunk, strict schema, nodes identified by name alone.

OntoGPT SPIRES 1.2.0. A LinkML template generated from the schema, one root class per article type, selected per collection. Whole document per call, default recursion into list items, no grounding annotators. OntoGPT splits lists on semicolons; following its own templates, the description of each list slot starts with "semicolon-separated list of". Each list item is extracted in a separate call that sees only the item, so the tenure of nearly every PlaysFor edge is *latest*.

	WUKONG s0 core	WUKONG s5	Neo4j	LlamaIndex	OntoGPT
LLM calls	4,011	4,447	2,044	2,295	2,728
Output tokens	2.44M	1.11M	1.37M	1.60M	0.39M
Wall time (min)	30	18	27	17	12
Edges	8,919	3,212	4,784	3,590	2,544
Entities found (of 216)	205	214	211	212	213
Entities split	62	8	40	23	60
Attribute values, exact (of 955)	449	871	589	242	618
Attribute values, lenient	763	871	817	340	770
Wrong values	47	21	21	11	56
Relations found (of 148)	133	136	141	142	139
Current team found (of 90)	18	77	45	1	82
precision	85%	90%	66%	100%	27%

Table 5: Schema-based extractors, one run each, next to WUKONG with the schema core only and with the revised full schema (means of three runs). For WUKONG s5 the current team is the document-level field; for the others, the teams on PlaysFor edges marked *latest*.

Notes.

- All three tools find the entities, and leniently most attribute values (Neo4j 817, OntoGPT 770), but fewer in the gold's form (589, 618), and they split 23–60 entities into several nodes. LlamaIndex fills few properties (340 lenient), and its name-only node identity merges entities of different types: it has 722 Team nodes, against 85 for WUKONG s5.
- OntoGPT finds 82 current teams because it marks 99% of its PlaysFor edges as *latest*.

6 Limitations

LightRAG, GraphRAG, KGGen and the three schema-based extractors were run once; WUKONG configurations three times. EDC (Extract-Define-Canonicalize, `baselines/edc_build.py`) is implemented and was piloted on 8 documents, but not run on the corpus: it makes one LLM verification call per extracted triple, and the pilot projects about 300,000 calls and 140M input tokens for its two modes, some 20 times a WUKONG run. Derived predicates come from labeling each edge independently, without a shared vocabulary, which is the situation of a user who wants to query these graphs by relation. s5 was designed after seeing the s4 results. The gold tables contain errors (mixed units for city areas, legal names for owners), which affect all systems alike.

Reproduction

From `paper/uda-nba/`: `run_full.sh` builds the LightRAG and GraphRAG graphs, `run_x6.sh` the WUKONG variants and `baselines/run_baselines.sh` the schema-based extractors and KGGen. Then `e1_analysis.py`, `derive_labels.py` (once per LightRAG, GraphRAG, KGGen), `relation_vocab.py` and `e1_table.py` give Table 2, and `x6_analysis.py` gives Tables 3 and 5. Data, environments and raw runs are not versioned; the derived results behind the tables are (`runs/e1/`, `runs/x6/`, `runs/labels/*.jsonl`).